

MOU

Silent Exclusion Diagnosis & Accountability Platform
for India's Public Distribution System (PDS / ONORC)

Software Requirements Specification

Version 2.0

Project	Mou
Document	Software Requirements Specification (SRS)
Version	2.0 — supersedes v1.0 (adds finalized tech stack, risk register, API contract)
Event	USAII Global AI Hackathon 2026 — Challenge Brief 4
Track / Direction	Undergraduate · Direction A (Benefits Navigator)
Build Window	June 14–21, 2026
Status	Build-ready

Revision History

Version	Date	Change
1.0	14 Jun 2026	Initial SRS: scope, three-stage architecture, FR-1..FR-20, responsible-AI controls, user journey.
2.0	14 Jun 2026	Finalized hybrid tech stack (Flutter + FastAPI + web dashboard); added explicit architecture & deployment section; promoted OCR quality to a first-class risk register with fallbacks; specified the FastAPI JSON API contract; resolved the officials-dashboard platform decision.

Table of Contents

1. Introduction

1.1 Purpose

This document specifies the software requirements for Mou, an AI-powered tool that helps legitimate beneficiaries of India's Public Distribution System (PDS) and the One Nation One Ration Card (ONORC) scheme understand why they are being silently excluded from their food entitlements, take a concrete next step to fix it, and — in aggregate — make the underlying systemic failures visible to authorities. Version 2.0 is the build-ready reference: it fixes the implementation architecture, names the technology stack, and states the project's dominant technical risk and its fallback path.

1.2 Scope

Mou addresses a documented failure mode of PDS/ONORC: a legitimate, enrolled beneficiary is silently cut off from rations because of a backend issue — a name or date-of-birth mismatch between Aadhaar and ration card (often across scripts, e.g. Bengali vs. English), an incomplete e-KYC / Aadhaar-seeding gap, or repeated biometric authentication failure. No notification is sent; the person discovers the exclusion only when turned away at the Fair Price Shop (FPS).

The product has two faces over one backend:

- Citizen-facing diagnosis (mobile app): the user submits documents and a short description; the system identifies the likely root cause and returns a plain-language explanation and a concrete next step (which office, which form).
- Accountability dashboard (web, for officials / civil society): each diagnosis becomes an anonymised event, clustered by root cause and location, surfacing systemic defects as a ranked, evidence-backed view.

Out of scope: Mou does not adjudicate eligibility, does not edit any government record, does not provide legal or medical advice, and does not guarantee outcomes. It diagnoses and flags; humans decide.

1.3 Definitions, Acronyms, and Abbreviations

Term	Meaning
PDS	Public Distribution System — India's subsidised food-grain distribution network.
ONORC	One Nation One Ration Card — ration portability across India via Aadhaar-linked cards.
FPS	Fair Price Shop — local ration shop where entitlements are collected.
e-KYC	Electronic Know Your Customer — Aadhaar-based identity verification linking beneficiary to ration card.
Seeding	Linking an Aadhaar number to a ration card record in the state system.
Silent exclusion	A legitimate beneficiary denied entitlement with no prior notice of the cause.

Term	Meaning
Root cause	The specific backend defect behind an exclusion (name mismatch, seeding gap, biometric failure, etc.).
Cluster	A group of distinct exclusion cases sharing root cause and location, treated as one systemic defect.
HITL	Human-in-the-loop — a decision deliberately left to a human rather than the AI.
API contract	The fixed JSON request/response shape between the frontends and the FastAPI backend.

2. Overall Description

2.1 Product Perspective

Existing government tools (state RCMS portals, the Mera Ration app) are per-user status lookups: they report whether a card is active or seeded, but not why it is failing, and never reveal that the same defect is affecting many others nearby. Mou is a diagnostic and accountability layer on top of this reality — it interprets the documents a beneficiary already holds, explains the failure in plain language, and aggregates failures into systemic signal. It uses publicly known scheme rules and the user's own documents; it requires no write-access to any government system.

2.2 Product Functions (Summary)

- Accept a beneficiary's documents (Aadhaar, ration card) and a short structured description of the problem.
- Extract identity fields via OCR, including non-Latin scripts (e.g. Bengali).
- Cross-match fields using transliteration-aware comparison to detect mismatches.
- Classify the exclusion into a root-cause category using deterministic rules.
- Return a plain-language diagnosis and a concrete next step.
- Record each diagnosis as an anonymised event and cluster events by root cause + location.
- Present a ranked dashboard of systemic defects ordered by beneficiaries affected.

2.3 User Classes and Characteristics

User class	Characteristics	Primary need
Beneficiary (self-serve)	Often low digital literacy, under stress, time-poor; may face a language/script barrier.	Know why they were excluded and what to do next.
Assisted user / proxy	Family member, CSC operator, or volunteer acting on the beneficiary's behalf.	Quickly diagnose for someone else.
Official / civil-society analyst	Block/circle-level officer or NGO worker with system context.	See where systemic exclusions cluster; prioritise action.

2.4 Design and Implementation Constraints

- Built entirely within the June 14–21, 2026 window; no pre-existing project code is reused as a deliverable.
- Eligibility / root-cause logic is implemented as deterministic, hardcoded rules (5–7 rules). No LLM sits on the eligibility decision path, eliminating hallucinated eligibility verdicts.
- LLM use (Gemini API) is confined to phrasing plain-language explanations, with pre-generated fallback explanations stored locally so a network failure cannot break the live demo.

- Synthetic documents and synthetic exclusion events are used for testing and for demonstrating the clustering layer at scale; no real beneficiary personal data is collected or stored.

2.5 Assumptions and Dependencies

- The user can supply reasonably legible photographs of their own Aadhaar and ration card.
- The OCR service reads Latin and at least one target Indic script (Bengali) to usable accuracy on real phone photos (see Risk R-1).
- Scheme rules used for diagnosis are stable for the demo scope (one scheme: PDS/ONORC).

3. System Architecture

Mou is one backend with two thin clients. The two frontends never communicate with each other; they are independent views over a single source of truth (the FastAPI service). This is the core stability property: adding the officials' dashboard cannot destabilise the citizen app, because the dashboard is read-only over the same JSON API and shares no mutable state with the app.

3.1 Components

Component	Responsibility
FastAPI service (Python)	Single source of truth. Hosts the OCR call, transliteration matching, hardcoded root-cause rules, Gemini explanation formatting, the event store, and cluster computation. Exposes the JSON API consumed by both clients.
Citizen app (Flutter)	Mobile app (APK). Captures documents + intake, calls /diagnose, renders side-by-side verification and result, calls /clusters for the “you are one of many” moment. Uses Firebase for offline support and app shell only.
Officials dashboard (web)	Read-only web app. Calls /clusters and renders ranked systemic defects with expandable underlying cases. Does not write data; does not depend on Firebase.
Datastore	Holds anonymised exclusion events. Owned by FastAPI; seeded with synthetic events for the clustering demo.

3.2 Data Flow

1. Citizen app sends documents + intake to FastAPI POST /diagnose.
2. FastAPI runs OCR → field extraction → transliteration-aware matching → rule-based classification → Gemini explanation (or local fallback), and writes an anonymised event to the datastore.
3. FastAPI returns the diagnosis JSON to the app, which shows the result and verification evidence.
4. Both the app and the dashboard call GET /clusters; FastAPI computes clusters once (groupby on event signatures) and returns the ranked list — guaranteeing both clients show identical numbers.

3.3 Stability Rationale

- Two views, one backend: the frontends share a fixed JSON contract, not a codebase. A dashboard failure leaves the app untouched.
- Dashboard is read-only: no write paths, no auth complexity for the demo, far fewer failure modes.
- Clustering computed once in FastAPI: avoids re-implementing groupby in two languages and prevents app/dashboard count mismatches.

- Parallelisable: Flutter (Dart) and dashboard (JS/TS) can be built independently against the contract from day 1 using mocked responses, with no blocking dependency on backend completion.

4. Technology Stack

Finalized hybrid stack. Every component is free-tier-capable, which also keeps the “Tools Used — free vs. paid” disclosure clean.

4.1 Stack by Layer

Layer	Technology	Why
Citizen app	Flutter (Dart)	Native APK deliverable, fast mobile-first UI, offline capability.
App data / shell	Firebase (Firestore + offline)	Offline cache and app shell only — not the event store.
Officials dashboard	Next.js / React (or Vite + React), Tailwind	Fast, polished, read-only web UI for data tables and expandable clusters.
Backend service	FastAPI (Python)	Python is where OCR, fuzzy matching, and deterministic rules live naturally; single microservice.
OCR	Google Cloud Vision API (Tesseract Bengali+English as fallback)	Reads Latin + Bengali script; free tier sufficient for demo volume.
Transliteration / matching	indic-transliteration + rapidfuzz	Romanise Bengali names, then fuzzy-match to catch e.g. Begum vs. Begam.
Root-cause classification	Plain Python rules (5–7 hardcoded)	Deterministic; no LLM on the eligibility path.
Explanation	Gemini API + local pre-generated fallbacks	Plain-language phrasing only; offline fallback protects the demo.
Event store	Datastore owned by FastAPI (Postgres on Render, or SQLite seeded with synthetic events)	Single source for cluster computation.
Deployment	FastAPI on Render/Railway; dashboard on Vercel; app as APK	All free-tier; clean separation of services.

4.2 Model / Tooling Roles (build assistance)

- DeepSeek V4 Pro (via Claude Code routing): bulk implementation — OCR wrapper, field extraction, matching code, clustering, API scaffolding.
- Opus: the reasoning-heavy parts — designing the 5–7 ONORC root-cause rules so they are credible to anyone who knows the system, and debugging transliteration matching when it fails to catch real mismatches.
- Google Stitch (optional, via MCP): generating polished UI drafts for the app flow and dashboard, exported to code and wired by the team.

Build-assistance tools are disclosed in the submission; their use does not touch the runtime eligibility path.

5. API Contract (FastAPI)

Lock this contract on day 1. Both frontends code against it in parallel using mocked responses, independent of backend completion.

5.1 POST /diagnose

Request (multipart): aadhaar_image, ration_card_image, symptom (enum), fps_location, language.

Response (JSON):

```
{
  "root_cause": "name_mismatch",          // enum
  "confidence": "high",                   // high | medium | low
  "extracted": {
    "aadhaar_name": "Rahima Begum",
    "ration_name_script": "রহিমা বেগম",
    "ration_name_romanized": "Rahima Begam"
  },
  "explanation": "Your name appears slightly differently ...",
  "next_step": { "office": "Circle Office", "form": "RC Correction" },
  "disclaimer": "Likely cause – verify before acting."
}
```

5.2 GET /clusters

Response (JSON): array of clusters, ranked by beneficiaries_affected (descending).

```
[
  {
    "root_cause": "name_mismatch",
    "fps_location": "Silchar FPS #4471",
    "beneficiaries_affected": 40,
    "confidence": "high",
    "cases": [ { "case_id": "anon-0192", "pattern": "Begum/Begam" }, ... ]
  }
]
```

6. Functional Requirements

Priority: *M* = must-have for demo, *S* = should-have, *C* = could-have.

6.1 Intake & Document Submission

ID	Requirement	Priority
FR-1	The app shall allow a user to upload images of an Aadhaar card and a ration card.	M
FR-2	The app shall let the user select a language and indicate self-serve vs. assisted (proxy) mode.	S
FR-3	The app shall capture a short structured symptom (e.g. turned away at FPS, card not found) and the FPS/location.	M
FR-4	The app shall detect unreadable / low-quality images and prompt re-upload.	S

6.2 Diagnosis Engine (FastAPI)

ID	Requirement	Priority
FR-5	The service shall extract identity fields (name, DOB, identifiers) from each document via OCR, including from at least one Indic script.	M
FR-6	The service shall perform transliteration-aware cross-document matching of names and key fields.	M
FR-7	The service shall classify the exclusion into exactly one primary root-cause category using deterministic, hardcoded rules.	M
FR-8	The response shall include the extracted fields from both documents so the app can show them side-by-side for verification.	M
FR-9	The service shall return a plain-language explanation of the likely cause and a concrete next step (office and form).	M
FR-10	The service shall express all diagnoses as “likely” / “may”, never as a definitive verdict.	M
FR-11	The service shall use a pre-generated fallback explanation when the live explanation service is unavailable.	M

6.3 Clustering & Accountability

ID	Requirement	Priority
FR-12	The service shall record each diagnosis as an anonymised event { root_cause, location, document-pattern }.	M
FR-13	The service shall cluster events sharing root cause and location into a single systemic defect.	M
FR-14	The service shall rank clusters by number of distinct beneficiaries affected.	M

ID	Requirement	Priority
FR-15	The dashboard shall display ranked clusters and allow each to expand to its anonymised underlying cases.	M
FR-16	The service shall only surface a cluster above a configurable confidence threshold.	S

6.4 Responsible-AI Controls

ID	Requirement	Priority
FR-17	The system shall never write to or modify any external government record.	M
FR-18	The system shall never output a definitive eligibility verdict; eligibility determination is reserved for a human official.	M
FR-19	The app shall let a user flag a diagnosis as incorrect, and shall log such feedback.	S
FR-20	The system shall display a clear disclaimer that Mou flags issues and does not provide legal advice or guaranteed outcomes.	M

7. Non-Functional Requirements

Category	Requirement	Target
Performance	End-to-end diagnosis latency from submission to result.	≤ 10 s typical
Usability	Core flow completable by a low-literacy user; plain language throughout.	≤ 4 steps
Accessibility	Self-serve, assisted/proxy, and a voice-or-large-text friendly path.	Demonstrated
Reliability	Live demo survives a network failure on the explanation path.	Offline fallback
Privacy	No real personal data persisted; stored events are anonymised.	By design
Transparency	Every diagnosis shows evidence; every cluster shows its cases.	Always
Portability	App runs as APK; dashboard runs in a standard browser.	Demonstrated

8. Risk Register

The dominant technical risk is OCR quality on real document photos (R-1). Transliteration matching is a bounded, known problem; OCR on phone photos of Indic-script ration cards is the genuine unknown, and it must be tested on day 1 before any matching code is written. Garbage OCR output makes even perfect matching logic useless.

ID	Risk	Severity	Mitigation / Fallback
R-1	OCR misreads names on real photos (esp. Bengali script), corrupting matching input.	High	Test Vision API on synthetic document images day 1. Preprocess (crop/contrast). Tesseract Bengali fallback. Worst case: demo on a curated high-quality pair and frame OCR as the demonstrated concept.
R-2	Misdiagnosis sends a beneficiary to the wrong office, costing scarce time.	Med	“Likely cause” framing; show extracted fields for user verification (FR-8/10); frame as a hypothesis to bring to the office, not a destination.
R-3	False cluster implicates an FPS/office unfairly.	Med	Confidence threshold before surfacing (FR-16); cases visible for audit (FR-15); human reviews before action.
R-4	Live API failure breaks the demo.	Med	Pre-generated explanations as local fallback (FR-11); pre-recorded screen-capture backup of the full flow.
R-5	Two-frontend scope overruns the window.	Med	Read-only dashboard (small); lock API contract day 1; parallelise Flutter and dashboard with mocked responses; Stitch for UI drafts.
R-6	ONORC rules oversimplified to the point of looking naive to informed judges.	Med	Opus-designed 5–7 rules grounded in real failure modes; cite real exclusion-error literature.

9. Responsible AI

Risk	Mitigation	Human-in-the-loop
Misdiagnosis sends a beneficiary to the wrong office.	“Likely cause” framing; extracted fields shown for verification before acting.	The user confirms the diagnosis against their real documents before acting.
A cluster falsely implicates an FPS / office.	Confidence threshold before surfacing; underlying cases visible for audit.	An official reviews the evidence before any action on a flagged pattern.
Over-reliance: user treats output as guaranteed eligibility.	No definitive verdicts; explicit disclaimer; never “you qualify.”	Eligibility and record correction remain with the issuing authority.

The decision Mou explicitly does NOT make: it never declares a person eligible/ineligible and never edits a live government record. A wrong automated correction could itself cause exclusion, and eligibility often turns on discretion the model cannot see — so that decision stays with a human official. Mou flags; it does not adjudicate.

10. Primary User Journey

The submission and demo are structured around one concrete person, then zoomed out to the systemic view.

10.1 Walkthrough — “Rahima Begum”

1. Rahima, a beneficiary in the Barak Valley, is turned away at her FPS with no explanation.
2. She (or a proxy) opens the Mou app, selects her language, and uploads her Aadhaar and ration card.
3. Mou extracts her name from both — English on Aadhaar, Bengali on the ration card — and detects a transliteration-level mismatch (Begum vs. Begam).
4. Mou classifies the cause as a name-mismatch seeding issue and explains, in plain language, why this silently blocks her ration.
5. Mou shows the two names side-by-side for her to verify, and gives the exact next step: which office, which form.
6. Her anonymised case joins the dashboard — revealing she is one of many at the same FPS hit by the same seeding-batch defect.

10.2 The Inversion

*One rejection is a personal misfortune. **Many rejections sharing one root cause at one shop is an accountability report the system cannot ignore — and Mou is what makes it visible.***

11. Demo Scope and Data Disclosure

11.1 Build-Window Scope

- One scheme (PDS/ONORC); one cross-script document pair (Bengali ration card + English Aadhaar).
- Stage 1 (diagnosis) fully live and working on a real scanned/photographed pair.
- Clustering + dashboard demonstrated on synthetic exclusion-event data, narrated as “every real diagnosis feeds this at scale.”
- Opening demo on a single named case, then zoom-out to the cluster.

11.2 Data Disclosure

- User-supplied document images (live diagnosis only; not persisted as personal data).
- Synthetic documents authored by the team with deliberate cross-script name/DOB mismatches.
- Synthetic, anonymised exclusion-event records generated to demonstrate clustering and ranking.

11.3 Out of Scope (this build)

- Integration with live government databases or write-back of corrections.
- Multiple schemes or Indic scripts beyond the demo pair.
- Accounts and production-grade security hardening.